Form Claw — Software Design Specification (SDS)

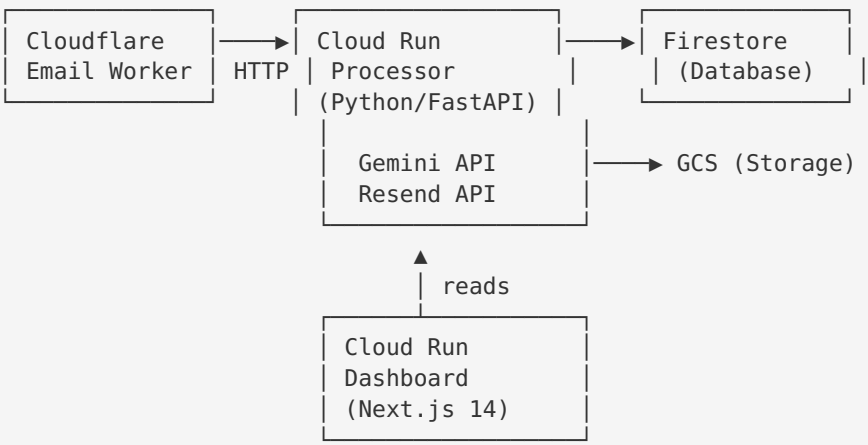
Version: 2.0
Date: July 2026
Project: Form Claw — Automated Hebrew PDF Form Filler

---

1. System Architecture

1.1 High-Level Architecture

Form Claw follows an event-driven microservices architecture with three independent components:



1.2 Component Communication

From	To	Protocol	Auth
Cloudflare Email Routing	Email Worker	Cloudflare internal	N/A
Email Worker	Processor	HTTPS POST	Bearer token
Processor	Firestore	gRPC	Service account
Processor	GCS	gRPC	Service account
Processor	Gemini API	HTTPS	API key
Processor	Resend API	HTTPS	API key
Dashboard	Firestore	gRPC	Service account
Dashboard	Cloudflare API	HTTPS	API token
Browser	Dashboard	HTTPS	Google SSO session

---

2. Component Design

2.1 Email Worker (`/email-worker`)

Technology: TypeScript on Cloudflare Workers runtime
Entry point: `src/index.ts`
Module: `postal-mime` for MIME parsing

Flow:

```
email(message, env, ctx)
    │
    ├─▶ Parse MIME with postal-mime
    ├─▶ Classify attachments (PDF / image / other)
    ├─▶ Extract threading headers
    │
    ├─ Has PDF/image? ─▶ POST /webhook (normal payload)
    └─ No valid files? ─▶ POST /webhook (intake_drop)
```

Attachment Classification:

- **PDF:** `application/pdf` MIME or `.pdf` extension
- **Image:** `image/jpeg` , `image/png` , `image/webp` , `image/heic` , `image/heif` or matching extensions
- **Other:** Logged and forwarded as `intake_drop`

2.2 Processor (`/processor`)

Technology: Python 3.11, FastAPI, uvicorn
Container: Cloud Run (Dockerfile)
Key dependencies: ReportLab, PyPDF2, pdf2image, pillow-heif, httpx

Module Structure:

```
processor/
├── main.py              # FastAPI app, webhook handler, LLM orchestration
├── llm_instructions.py  # Centralized Gemini prompts
├── form_filler.py       # Code executor + asset path rewriting
├── security_filter.py   # Prompt injection detection
├── family_data.json     # Family member data
├── requirements.txt     # Python dependencies
├── Dockerfile           # Container image definition
└── assets/
    ├── fonts/           # Hebrew + English fonts
    └── signatures/      # Family signature PNGs
```

Processing Pipeline (detailed):

```
POST /webhook
    │
    ├─▶ 1. Auth check (Bearer token)
    ├─▶ 2. Route: intake_drop? → _handle_intake_drop()
    ├─▶ 3. Create Firestore log document
    ├─▶ 4. Security scan (subject + body + filenames)
    │       └─ Blocked? → raise ValueError
    ├─▶ 5. Extract & classify attachments
    │       ├─ PDF found → use directly
    │       └─ Images only → images_to_pdf()
    ├─▶ 6. PDF → page images (pdf2image, 200 DPI)
    ├─▶ 7. analyze_form() → Gemini vision
    │       ├─ System prompt: FORM_ANALYSIS_SYSTEM
    │       └─ User prompt: build_analysis_prompt()
    ├─▶ 8. extract_target_person()
    ├─▶ 9. generate_fill_code() → Gemini code gen
    │       ├─ System prompt: CODE_GENERATION_SYSTEM
    │       ├─ User prompt: build_code_generation_prompt()
    │       └─ Includes: family_data + knowledge entries
    ├─▶ 10. execute_fill_code()
    │       ├─ rewrite_asset_paths() → container paths
    │       ├─ exec() in restricted namespace
    │       └─ Call fill_form(pdf_bytes, family_data)
    ├─▶ 11. Upload filled PDF to GCS
    ├─▶ 12. Reply via Resend (with threading headers)
    └─▶ 13. Update Firestore log (success/failure)
```

LLM Instructions (`llm_instructions.py`):

Prompts are maintained as a dedicated Python module with:
- **System prompts** — Define the AI's role and capabilities
- **Prompt builders** — Functions that construct user prompts with dynamic data
- **Structured output** — JSON schema for form analysis, Python code for filling

Two prompt sets:
1. **Form Analysis** — Vision analysis of form images, outputs field map JSON
2. **Code Generation** — Generates ReportLab/PyPDF2 fill code with 12 detailed rules

Security Filter (`security_filter.py`):

- Pattern-based detection with weighted risk scoring
- Threshold: 70% risk score = blocked
- Categories: direct override, role hijack, exfiltration, form attack, code injection
- Hebrew variant patterns included
- Scans: subject, body, attachment filenames

Code Executor (`form_filler.py`):

- Executes LLM-generated Python in controlled namespace
- Rewrites hardcoded paths to container-local assets
- Validates output type (must return bytes)
- Available imports: io, os, sys + ReportLab + PyPDF2

2.3 Dashboard (`/dashboard`)

Technology: Next.js 14 (App Router), TypeScript, Tailwind CSS
Auth: NextAuth.js with Google SSO provider
Data: Firestore (direct client, not Prisma)

Page Structure:

```
/                 → Redirect to /dashboard
/login            → Google SSO login
/dashboard        → Overview (stats, recent activity)
/activity         → Processing log (search, filter, pagination)
/statistics       → Charts (daily volume, success rate, timing)
/errors           → Error logs with CSV export
/knowledge        → Knowledge base CRUD
/system           → System health, email intake, E2E tests
/settings         → User settings
```

API Routes:

Route	Method	Purpose
`/api/auth/[...nextauth]`	GET/POST	NextAuth.js handlers
`/api/e2e-test`	POST	Pipeline health checks
`/api/intake`	GET	Cloudflare worker analytics

---

3. Data Design

3.1 Firestore Collections

`form_processing_logs`

Field	Type	Description
received_at	timestamp	When the email was received
email_message_id	string	Original email Message-ID
sender_email	string	Sender email address
subject	string	Email subject line
processing_status	string	`processing`, `success`, `failed`, `dropped`
target_person	string	Detected target person
attachment_filename	string	Original attachment filename
attachment_count	number	Number of attachments
source_type	string	`pdf` or `image`
converted_from_images	boolean	Whether images were converted to PDF
image_count	number	Number of images converted
page_count	number	Number of PDF pages
filled_pdf_path	string	GCS path to filled PDF
processing_time_seconds	number	Total processing time
processing_completed_at	timestamp	Completion timestamp
llm_provider	string	Model used (e.g., `google/gemini-flash-latest`)
error_message	string	Error details (if failed)
error_type	string	Error class name (if failed)
attachment_summary	array	Attachment metadata (for drops)

`knowledge_entries`

Field	Type	Description
key	string	Entry identifier
value	string	Entry content
category	string	Category (medical, educational, etc.)
applies_to_person	string	Person name or “Family-wide”
is_active	boolean	Whether entry is active
created_at	timestamp	Creation timestamp
updated_at	timestamp	Last update timestamp

`system` (single document: `current`)

Field	Type	Description
webhook_enabled	boolean	Whether to process incoming emails
last_health_check	timestamp	Last health check timestamp

3.2 Cloud Storage Structure

```
formclaw-assets/
├── config/family_data.json    # Fallback family data
├── fonts/FtPilKahol2.ttf      # Hebrew font
├── fonts/Playzone.ttf          # English font
├── signatures/zeev_signature.png
├── signatures/keren_signature.png
└── filled/{log_id}_{filename}.pdf
```

---

4. Deployment Design

4.1 CI/CD Pipeline

GitHub Actions with three parallel deployment workflows:

```
push to main
    │
    ├─ processor/** changed → deploy-processor.yml
    │       ├─ Build container → Cloud Run
    │       └─ E2E smoke test
    │
    ├─ dashboard/** changed → deploy-dashboard.yml
    │       ├─ Build container → Cloud Run
    │       └─ E2E smoke test
    │
    └─ email-worker/** changed → deploy-email-worker.yml
            ├─ npm install → wrangler deploy
            └─ E2E smoke test
```

4.2 E2E Smoke Test (`e2e-test.yml`)

Reusable workflow called after each deployment:
1. Wait 30s for deployment to settle
2. Check processor `/health` endpoint
3. Verify webhook endpoint reachability
4. Report pass/fail summary

4.3 Infrastructure Configuration

Component	Config
Processor	1 CPU, 1Gi RAM, 300s timeout, 0-3 instances
Dashboard	1 CPU, 512Mi RAM, 0-2 instances
Email Worker	Cloudflare Workers free tier
GCP Auth	Workload Identity Federation (no service account keys in CI)

---

5. Error Handling

5.1 Error Categories

Category	Handler	User Impact
No valid attachments	`intake_drop`	Reply with instructions
Security filter block	ValueError	Reply with rejection
Gemini API failure	500 + error reply	Retry suggestion
Code generation error	500 + error reply	Retry suggestion
Code execution error	RuntimeError + error reply	Retry suggestion
Resend failure	Logged, no reply	Silent failure

5.2 Error Flow

```
Exception caught
    │
    ├─▶ Log to Firestore (status: failed, error details)
    ├─▶ Send error reply to sender via Resend
    └─▶ Return HTTP 500 to caller
```

---

6. Future Considerations

- **Multi-attempt filling** — Retry with adjusted prompts on first failure
- **Form template caching** — Cache analysis for recurring form types
- **Confidence scoring** — Rate filling quality before sending
- **OCR validation** — Re-read filled form to verify correctness
- **Batch processing** — Handle multiple forms in a single email
- **WhatsApp integration** — Accept forms via WhatsApp messages